

作业 2:Tensor Core & Pipeline

Weiming HPC Training Camp × LCPU AI Infra Seminars • Session 3 / 4

Preface

这次作业主要是 Session 3 的配套练习，内容围绕 Tensor Core 展开，其中 Module 4 还会用到 Session 4 介绍的 tiling、TMA 和 pipeline。完成作业时主要参考课程课件和 PTX ISA 文档。

题型沿用系列惯例，并新增 DERIVE: 先手工推导，再写程序验证推导。共七种: CONCEPT 概念题、DERIVE 推导题、MODIFY 改造题、DEBUG 修 bug 题、EXPERIMENT 实验题、HANDS-ON 动手题、FROM-SCRATCH 从零实现题。标 Optional 的为选做。涉及代码的题在标题行右侧标出文件路径 (相对 assignment02/)，题面细节以文件头注释为准；FROM-SCRATCH 题。

硬件与构建: 主线环境是 B300, cuda/Makefile 默认 ARCH=100f; M0、M1 也可以在 5090 上完成 (ARCH=120a `make ...`); M2 的判测纯 host, 无卡可判; M3、M4、M5 需要 B300。本作业统一用显式 `-gencode` 而不用 `-arch=sm_XXXa` 简写，原因见 assignment02/README.md, Makefile 已经配好。

实验纪律: 所有性能数字都在自己占满的卡上测；测前测后 `nvidia-smi --query-compute-apps=pid, name --format=csv` 查有没有别的进程占卡；可能 hang 的程序套 `timeout -k 5 <秒数>` (判测脚本已带)。

AI 政策: 必做题沿用仓库根目录 CLAUDE.md——AI 可以帮你理解、review，不能替你实现；团队题不设限制。详见 assignment02/README.md。

Content

Module	主题	对应课件	代码位置
0	环境与峰值	P1(S008–S021)	cuda/m0_env/
1	sm80:fragment 与 mma.sync	P2.2(S025–S041)	cuda/m1_sm80/
2	smem 供数:descriptor 与 swizzle	P2.3(S042–S070)	cuda/m2_smem/
3	sm100:tcgen05	P2.4(S071–S093)	cuda/m3_tcgen05/
4	完整 GEMM 四步走	S084 + Session 4	cuda/m4_gemm/
5	低精度与 block scaling	P3(S094–S111)	cuda/m5_lowprec/ kernels/
6	TileLang 对照	收尾 (S112)	(复用 assignment01)
Team	FlashKDA / MSA decode	—	team/

0 环境与峰值

先确认工具链和卡都能发出 tensor core 指令：编译一个最小 tensor core 程序，测试卡的峰值性能，后面就可以用它来计算各个实现的性能达成率。

参考资料

Session3 课件 S008-S021；PTX ISA 的 mma 指令一节 (形状与 dtype 表)；你手里每块卡的官方 datasheet(或 whitepaper)。

prob 0.1 (HANDS-ON)

cuda/m0_env/01_first_mma.cu

编译并运行最小 Tensor Core 程序，它使用一个 warp 执行一条 m16n8k16 的 mma.sync 指令，并与 CPU 计算结果进行比较。

```
cd assignment02/cuda
make run/m0_env/01_first_mma
```

在你能使用的 GPU 上分别运行该程序（5090 使用 ARCH=120a make ..., B300 使用默认配置）。尝试使用不匹配的 ARCH 编译运行一次，记录现象，并结合 assignment01 Module 8 中 fatbin/JIT 的内容解释原因。可使用 make ptx/m0_env/01_first_mma 查看生成的 PTX。

prob 0.2 (DERIVE)

推导你所使用 GPU 的 Tensor Core 理论峰值。参考课上 A100 的推导方法（S018–S019），分别计算 5090 和 B300 的 bf16 峰值，并根据 dtype 宽度关系估算 fp8 / fp4 峰值。

开始计算前先明确采用的口径，包括 dense 或 sparse、boost 或 base 频率、FMA 是否计作 2 FLOP 等。完成推导后，再与 datasheet 中的官方数据进行对照；如果结果存在差异，需要说明差异来自哪一项口径。

量	5090	B300
bf16 FLOP/cycle/SM		
bf16 峰值 (TFLOPS)		
fp8 峰值 (TFLOPS)		
fp4 峰值 (TFLOPS)		
datasheet 对照值与口径差异		
HBM/GDDR 带宽 (GB/s)		
机器平衡点 (FLOP/byte, bf16)		

根据 bf16 峰值和显存带宽计算机器平衡点 (FLOP/byte)，并与单条 mma 的计算强度 (S016, m16n8k16 fp16 为 3.2 FLOP/byte) 比较。思考两者之间的差距意味着什么，以及为什么后续 M2–M4 需要从数据供给路径入手优化。

prob 0.3 (CONCEPT)

判断下列说法是否正确，并给出一句理由。

- (a) 一条 mma 的计算强度，分子是 $2MNK$ ，分母按 A、B 读入与 D 写回的字节总和计 (S016 的口径)。

- (b) `mma.sync` 是 warp 级协作指令:32 个 lane 各持 fragment 的一部分, 要求全 warp 一致地执行这条指令; 有 lane 发散时行为未定义。
- (c) 增大 mma 的形状 M/N/K 能提高单条指令的计算强度, 而且没有代价, 所以指令形状越大越好。
- (d) 只要单条 mma 的计算强度低于机器平衡点, GEMM kernel 就不可能逼近计算峰值。

1 sm80:fragment 与 mma.sync

课上对 m16n8k16 fp16 推过 fragment 公式、手搓过单 tile mma (C03–C07)。现在你手推一遍这个 m16n8k32 fp8 shape, 再看 `ldmatrix` 到底发挥了什么作用。

参考资料

课件 S025–S041、C03–C08; PTX ISA 的 “Matrix Fragments for `mma.m16n8k32`” 一节与 “Warp-level matrix load instruction: `ldmatrix`” 一节; fp8 转换用 `cuda_fp8.h(__nv_fp8_e4m3)`。

prob 1.1 (DERIVE)

`cuda/m1_sm80/01_fragment_map.cu`

根据 PTX 文档中的 fragment 布局, 推导 `mma.sync.aligned.m16n8k32.row.col.f32.e4m3.e4m3.f32` 对应的 A、B fragment 映射公式, 并完成文件中的四个函数。判测使用纯 host 的 32-lane 真值表比对, 无需 GPU 即可完成:

```
cd assignment02/cuda
make run/m1_sm80/01_fragment_map
```

附加问题 (写入报告): A 的同一个 b32 寄存器中的 4 个 fp8 元素沿矩阵哪个方向相邻? 这个布局对 1.4 中使用 `ldmatrix load` 有什么影响?

prob 1.2 (DEBUG)

`cuda/m1_sm80/02_bug_fragment.cu`

这个程序发一条 m16n8k16 fp16 mma, 判测会 FAIL。先运行一遍, 后改动:

- (a) 描述症状: D 的哪些位置错、错成了什么 (和对的部分是什么关系);
- (b) 修好它, 并解释错的是哪个 fragment 的哪部分映射, 为什么恰好产生 (a) 的症状。

```
cd assignment02/cuda
make run/m1_sm80/02_bug_fragment
```

prob 1.3(FROM-SCRATCH): 手写单 tile fp8 mma

在 `cuda/m1_sm80/03_mma_fp8.cu` 中从零实现一个单 tile 的 fp8 mma, 不提供代码骨架。使用 m16n8k32 e4m3 mma、f32 累加, 并手动完成 fragment 装载 (本题不使用 `ldmatrix`), 最后与 CPU 参考结果进行严格相等比较。

要求如下:

输入使用小整数, 保证转换为 e4m3 后可以精确表示;

程序接受一个 seed 参数, 例如 `./prog 123`;

输出必须以 PASS 或 MISMATCH / FAIL 开头;

fp8 与 float 的互转直接使用 `cuda_fp8.h`。

1.1 中推导的 fragment 映射公式会直接用在这里。映射只要有一处错误，随机数据的判测就无法通过。

判测脚本会使用五个 seed，全部通过才算完成：

```
cd assignment02/cuda/m1_sm80
./judge_mma_fp8.sh 03_mma_fp8.cu
```

prob 1.4 (MODIFY)

cuda/m1_sm80/04_ldmatrix.cu

在给定骨架中保留 1.3 的手工装载方式，并另外实现一条使用 ldmatrix 的装载路径。两种实现需要共存，并分别通过判测。

根据 PTX 文档选择合适的 ldmatrix 变体（.x1/.x2/.x4，以及是否使用 .trans）。B 矩阵在 shared memory 中的布局还需要满足 ldmatrix 的 16 byte 行地址要求，具体约束见文件头说明。

两条路径都通过判测后，使用 `make ptx/m1_sm80/04_ldmatrix` 或 `nvdiasm` 查看生成的指令，分别统计 smem → fragment 阶段的装载指令和地址计算指令数量，并回答：

- (a) ldmatrix 省掉了手工装载中的哪些工作？
- (b) 为什么这些工作在手工装载路径中无法避免？

prob 1.5 (EXPERIMENT)

cuda/m1_sm80/05_ldsm_stride.cu

观察行跨度对 ldmatrix 的影响。对同一个 16×16 fp16 tile，分别使用 32 B、64 B、128 B 和 128+16 B（padding）四种行跨度。

先根据课上介绍的 bank 模型，预测四种情况下执行一次 ldmatrix 所需的 wavefront 数及其比例，再运行程序并使用 Nsight Compute 测量：

```
cd assignment02/cuda
make run/m1_sm80/05_ldsm_stride
ncu --metrics l1tex__data_pipe_lsu_wavefronts_mem_shared_op_ld.sum,
    l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum ./bin/m1_sm80/05
    _ldsm_stride
```

档位	预测 wavefront 比	实测 wavefront	实测 conflict	平均 cycle
32 B				
64 B				
128 B				
128+16 B				

比较预测与实测结果：哪一种行跨度使 wavefront 数增加到 4 倍？wavefront 的比例应与 bank 模型一致，但实际耗时的差距通常没有这么大。结合 8 个 warp 的占用情况，解释为什么 wavefront 增加 4 倍并不会使总耗时也增加 4 倍。

Editor's Note

1.3–1.5 关注的是同一个问题：怎样把数据从 shared memory 装入 Tensor Core 的 fragment。1.3 手动完成每个 lane 的装载，1.4 使用 `ldmatrix` 简化这一步，而 1.5 说明即使使用 `ldmatrix`，shared memory 的 bank conflict 仍然会影响实际效率。

后面的模块会继续沿着这条数据供给路径展开：M2 中使用 descriptor 描述布局，M4 中进一步使用 TMA 和 pipeline 完成数据搬运。

2 smem 供数:descriptor 与 swizzle

sm90 起，tensor core 从 smem 取数不再经过 fragment 装载指令，而是读一个 64 位 descriptor 按 canonical 布局自取。descriptor 与 swizzle 的格式在 sm100 的 tcgen05 上原样沿用，本模块推的每个公式都是 M3、M4 的直接组件。

参考资料

课件 S042–S070(proxy:S051; core matrix 与 LBO/SBO:S057–S061; swizzle:S062–S065); PTX ISA 的 “Asynchronous Warpgroup MMA Shared Memory Layout” 与 swizzling 小节。

prob 2.1 (CONCEPT)

(a) 一个 warpgroup 使用 `wgmma` 读取刚写入 shared memory 的数据。将下面六个操作排成正确顺序，并说明每一步用于避免哪两个参与者之间的哪种乱序：

```
wgmma.mma_async / st.shared / wgmma.commit_group / fence.proxy.async / wgmma.fence /  
wgmma.wait_group
```

(b) 判断下列说法是否正确，并给出一句理由。

1. `fence.proxy.async` 是 `wgmma` 专属的指令，TMA 与 tcgen05 的场景不需要它。
2. `wgmma.commit_group` 会阻塞，直到它之前发射的 `wgmma` 全部完成。
3. 不加 `fence.proxy.async` 时，`wgmma` 可能读到 shared memory 中的旧值，因为 `st.shared` 的写经过 generic proxy，而 `wgmma` 的读经过 async proxy。

prob 2.2 (DERIVE)

`cuda/m2_smem/02_descriptor.cu`

根据文件头给出的位域，实现 SM100 的 64 位 smem matrix descriptor 编码函数，并分别对下面三种情况推导 LBO、SBO 和 layout：

- K-major, 无 swizzle;
- K-major, 128B swizzle;
- MN-major, 128B swizzle。

判测为纯 host，无需 GPU。测试使用的描述符真值已经在 B300 上通过实际 tcgen05 指令验证，3.2 中也会使用这三组描述符：

```
cd assignment02/cuda  
make run/m2_smem/02_descriptor
```

场景 2 和场景 3 最终得到的 descriptor 相同。在报告中回答：MN-major 与 K-major 的区别体现在哪里？

prob 2.3 (FROM-SCRATCH)

cuda/m2_smem/03_swizzle.cu

实现 128B、64B 和 32B 三种 swizzle 模式的地址映射函数，将逻辑坐标映射到 atom 内的物理字节偏移。

课件 S064 已经推导了 128B swizzle 的地址位异或关系；64B 和 32B 两种模式需要根据 PTX ISA 的 swizzling 一节自行推导。

判测分为两部分：首先检查映射是否为双射，然后检查列访问是否存在 bank conflict。单纯使用 padding 或恒等映射虽然可能通过第一项，但无法通过第二项。判测同样为纯 host：

```
cd assignment02/cuda
make run/m2_smem/03_swizzle
```

本题实现的 swizzle_128B 会在 3.2 中直接用于 shared memory staging，并接受实际硬件上的 GEMM 判测。若 3.2 或 4.1 出现问题，可以先运行本题的判测，排除 swizzle 布局错误。

Editor's Note

课件 C12–C14 给出了使用 wgmma 实现单 tile 的完整示例，本作业不单独设置对应题目。wgmma 是 sm_90a 专属指令，5090 (sm120) 和 B300 (sm100) 均无法运行；有兴趣的同学可以在集群 H 卡上自行尝试。

本模块重点保留 wgmma 路径中可以继续使用的两个部分：descriptor 与 swizzle。2.2、2.3 先分别完成它们的推导与判测，后面的 M3 会在实际硬件上继续使用。

3 sm100: tcgen05

在 sm100 上，tcgen05 将累加器放入 TMEM，并使用 mbarrier 处理异步完成通知。本模块会先在 B300 上完成一个单 tile GEMM，再通过后续实验理解 TMEM、mbarrier 和 2-CTA MMA 的使用方式。

参考资料

课件 S071–S093 (TMEM: S073–S075; mma 与 idesc: S076–S079; mbarrier: S080–S084; ld 与同步: S085–S086; 七步流程: S087/F27; 2-CTA: S091); PTX ISA 的 tcgen05 一族小节 (alloc / mma / commit / ld / fence) 与 mbarrier 一节。

prob 3.1 (CONCEPT)

判断下列说法是否正确，并给出一句理由。其中 (d) 需要写出计算过程。

- (a) tcgen05.ld 读取 TMEM 时，每个 warp 只能读取自己对应的 32 条 lane，warp 之间不能互相读取。
- (b) 与 mma.sync 由 warp 协作、wgmma 由 warpgroup 协作不同，tcgen05.mma 由单个线程发射，随后由硬件异步执行。
- (c) TMEM 中的累加结果可以直接通过 TMA 搬回 global memory，不需要经过寄存器。

- (d) TMEM 每个 SM 包含 $128 \text{ lane} \times 512 \text{ column} \times 4 \text{ B}$; 一个 m128n256 的 f32 accumulator 恰好占用其中一半。
- (e) tcgen05.commit 会阻塞直到之前发射的 mma 全部完成, 因此 commit 返回后即可安全读取 TMEM。

prob 3.2(FROM-SCRATCH):tcgen05 单 tile GEMM

cuda/m3_tcgen05/02_single_tile.cu

从零实现一个 tcgen05 单 tile GEMM: 计算 m128n64k64 的 bf16 矩阵乘, 使用 f32 累加, cta_group::1, 并由一个 128 线程的 block 完成。

数据按照下面的路径流动:

global $\rightarrow$ smem (K-major + 128B swizzle) $\rightarrow$ tcgen05.mma $\rightarrow$ TMEM $\rightarrow$ tcgen05.ld $\rightarrow$ global

代码骨架只保留课件 F27 中的七步流程注释。descriptor 使用 2.2 中实现的编码方式, shared memory staging 使用 2.3 中的 swizzle_128B。TMEM alloc、mbarrier、idesc 以及 mma / ld 的具体写法需要根据 PTX ISA 完成, 相关语义可参考课件 C15–C21。

fence.proxy.async 的位置以及 tcgen05.ld 的 warp 可见范围已经在文件头中给出。

判测:

```
cd assignment02/cuda
make run/m3_tcgen05/02_single_tile
cd m3_tcgen05 && ./judge_tile.sh
```

在正确版本通过后, 故意去掉 fence.proxy.async 再运行一次, 记录出现的现象并写入报告。结合 2.1(a) 中的排序问题解释原因。

prob 3.3 (DEBUG)

cuda/m3_tcgen05/03_bug_mbarrier.cu

这个程序连续发射多轮 mma, 并在每一轮后读取结果, 用来模拟 M4 中沿 K 维流式计算的过程。当前程序的 mbarrier 使用存在错误, 判测带有超时机制, 因此程序挂死本身也是需要观察的现象。

先运行:

```
cd assignment02/cuda
make bin/m3_tcgen05/03_bug_mbarrier
cd m3_tcgen05 && ./judge_mbar.sh
```

- (a) 分别记录 rounds = 1、2、4 时的运行结果以及问题出现的条件。
- (b) 修复程序, 并画出修改前后 mbarrier 的状态变化, 包括 phase 和 arrival count。指出错误版本在哪一次等待中过早或过晚放行, 并解释为什么会产生 (a) 中观察到的现象。

提示: 注意 tcgen05.ld 与尚未完成的 mma 之间的关系。

prob 3.4 (EXPERIMENT)

cuda/m3_tcgen05/04_cta_pair.cu

比较 cta_group::1 和 cta_group::2 完成同一个 m256n64k64 任务时的数据开销。

cta_group::1 使用两个独立 block; cta_group::2 使用一个 cluster 发射一条 M=256 的 mma, 两个 CTA 分别保存 B 矩阵的一部分。

先预测, 再运行实验:

- (a) 在 `cta_group::2` 中，每个 CTA 所需的 B shared memory 是 `cta_group::1` 的多少？TMEM 的占用又如何变化？程序会打印 shared memory 用量，用它验证你的预测。
- (b) 使用 Nsight Compute 比较两种实现的 shared memory 总流量。
- (c) `cta_group::2` 节省下来的 shared memory 容量，在 M4 的 pipeline 中可以用来做什么？
- (d) `cta_group::2` 依赖 sm90 引入的哪一种硬件机制？5090 不支持 2-CTA MMA，结合 0.2 中整理的硬件参数，说明为什么这类机制更常出现在数据中心 GPU 上。

运行：

```
cd assignment02/cuda
make run/m3_tcgen05/04_cta_pair
```

在这个单 tile 实验中，两种实现的运行时间差异处于噪声范围内，因此不要用耗时判断优劣。主要比较 shared memory 用量以及 Nsight Compute 测得的流量。

4 完整 GEMM

本模块将 3.2 的单 tile 实现扩展为完整的 B300 GEMM，并依次加入 tiling、TMA 和多级 pipeline。实验统一使用 4096³ 的 bf16 GEMM，tile 大小固定为 128×64×64，并与 cuBLAS 结果进行严格相等比较。

参考资料

课件 S084 (pipeline 骨架)、S087; Session 4 讲义对应章节; PTX ISA 的 `cp.async.bulk.tensor`、`mbarrier (expect_tx)` 小节; CUDA Driver API 的 `cuTensorMapEncodeTiled`。

下面的表格贯穿 4.1–4.3。第 0 行需要在同一块 GPU 上重新运行 assignment01 Bonus 中的 naive matmul。由于该实现使用 fp32，只比较性能量级。

实现	TFLOPS	对 cuBLAS 达成率	一句话：时间主要花在哪
naive (assignment01, fp32)			
4.1 tiled			
4.2 TMA			
4.3 pipeline (S=3)			
cuBLAS		100%	

prob 4.1 (FROM-SCRATCH)

`cuda/m4_gemm/01_tiled.cu`

将 3.2 的单 tile 实现扩展为完整 GEMM：使用 grid 覆盖所有输出 tile，并沿 K 维循环完成累加。数据 staging 仍然使用 `st.shared` 和 `swizzle`。

相对 3.2，本题主要新增 tile 映射以及 K 循环中的同步和累加。具体步骤见文件头。

```
cd assignment02/cuda
make run/m4_gemm/01_tiled
```

通过判测后，填写性能表中 4.1 tiled 一行。结合 0.2 中计算的机器平衡点，判断此时性能主要受哪一环节限制。

prob 4.2 (MODIFY)

cuda/m4_gemm/02_tma.cu

将 4.1 中的 shared memory staging 改为 TMA。

host 端使用 `cuTensorMapEncodeTiled` 创建 tensor map; kernel 中使用 `cp.async.bulk.tensor` 搭配 `mbarrier` 的 `expect_tx` 完成搬运。本题仍然使用单缓冲。

tensor map 的参数以及同步方式的变化已经在文件头中给出。swizzle 由 TMA 根据 tensor map 自动完成, 原有 descriptor 不需要修改字段。

```
cd assignment02/cuda
make run/m4_gemm/02_tma
```

通过判测后, 填写性能表中 4.2 TMA 一行。

比较 4.1 与 4.2, 并回答: 4.1 中 shared memory staging 的开销由哪些部分组成? 改用 TMA 后, 其中哪些工作不再由普通 CUDA 指令完成?

使用 Nsight Compute 辅助分析, 观察 4.1 中 SM 时间主要消耗在哪些部分。

prob 4.3(FROM-SCRATCH): 多级流水

cuda/m4_gemm/03_pipeline.cu

将 4.2 的单缓冲改为 STAGES 级循环缓冲, 使后续 K tile 的 TMA 预取能够与当前 tile 的 mma 计算重叠。

STAGES 为编译参数, 例如:

```
STAGES=4 make -B run/m4_gemm/03_pipeline
```

这里的 `-B` 不能省略, 否则修改 STAGES 后可能不会重新编译。

文件头给出了每个 stage 使用双 `mbarrier` 的基本结构, 同时包含一个需要处理的 pipeline hazard: 强制发射与机会式预取之间的边界处理错误会导致死锁。在该错误下, 1024^3 可能偶尔通过, 而 4096^3 会稳定暴露问题。开始实现前先阅读文件头说明。

本题不要求 warp specialization、persistent kernel 或 epilogue 融合, 也不设置 cuBLAS 达成率门槛。重点是流水实现是否正确, 以及能否根据实验结果解释性能变化。

```
cd assignment02/cuda
make run/m4_gemm/03_pipeline
cd m4_gemm && ./sweep_stages.sh
```

完成以下内容:

1. 使用 $S=3$ 的结果填写性能表中 4.3 pipeline 一行。
2. 完成不同 stage 数的性能测试:

形状	S=2	S=3	S=4	S=6
4096^3				
$256 \times 4096 \times 16384$				

比较两个形状对 STAGES 的敏感程度，并结合 shared memory 用量、每个 SM 可同时驻留的 block 数以及 block 间并发能够隐藏的延迟进行解释。

3. 任选一个 STAGES，画出稳态阶段各 stage 中 TMA 与 mma 的流水时空图。

4. 回答：

(a) 从 4.1 到 4.3，主要瓶颈发生了哪些变化？

(b) 梯子表中每一级优化分别减少了哪部分开销？

(c) 如果继续增大 tile 或增加 stage 数，shared memory 与 TMEM 哪一个会先成为容量限制？结合 3.4(c) 的结果说明。

prob 4.4 (FROM-SCRATCH • Optional)

任选一个方向继续优化：

(a) 实现 4.3 的 `cta_group::2` 版本，利用 B shared memory 用量的减少增加 pipeline 深度或扩大 tile；

(b) 在 5090 上使用 `mma.sync` 实现相同的 tiling，并与 B300 的结果比较；

(c) 自由优化当前 kernel，提高相对 cuBLAS 的性能，并记录每一步优化解决了什么问题。

prob 4.5 (EXPERIMENT)

`cuda/m4_gemm/05_thin_gemm.cu`

观察矩阵形状较窄时 Tensor Core GEMM 的性能变化。

实验形状取自 vLLM 主树中 Kimi K3 的 decode GEMM dispatch 表：

`vllm/models/kimi_k3/nvidia/low_latency_gemm.py`

使用其中七个投影层对应的 N、K，并对 M 取：

`{1, 8, 16, 64, 256, 1024, 4096, 16384, 65536}`

其中 N 和 K 由权重形状决定，变化的 M 表示一次 GEMM 处理的 token 数量。较小的 M 对应 decode batch；较大的 M 可以对应 chunked prefill 中单次处理的 chunk。vLLM 在 $M \leq 16$ 时会放弃 cuBLAS / Tensor Core 路径，改用 CUDA Core FMA 的 skinny kernel。

运行程序前，先对每个形状计算 arithmetic intensity：

$$AI = \frac{2MNK}{2MK + 2NK + 2MN}$$

将结果与 0.2 中得到的机器平衡点比较，并计算对应的理论性能上限：

- compute bound：Tensor Core 峰值；
- memory bound： $AI \times$ 显存带宽。

然后运行：

```
cd assignment02/cuda
make bin/m4_gemm/05_thin_gemm
./bin/m4_gemm/05_thin_gemm <峰值TFLOPS> <带宽GB/s>
```

其中两个参数使用 0.2 中得到的数值。

程序会输出 TFLOPS、GB/s、AI 以及相对于 compute roof 和 memory roof 的达成率。根据结果回答：

(a) 描述性能随 M 变化的趋势。M 较小时，Tensor Core 性能从什么时候开始明显下降？M 增大到什么范围后进入相对稳定的平台？平台上的达成率是多少？

- (b) 对性能下降的形状，比较 compute roof 和 memory roof 两种达成率。哪些形状主要受到显存带宽限制？
- (c) `f_b_proj` ($K=128$) 中两个 roof 的达成率都较低。结合矩阵形状分析，它的限制来自哪里？
- (d) 根据上述结果解释 vLLM 在 $M \leq 16$ 时选择 skinny CUDA Core kernel 的原因。

`in_proj_qkvfab` 对应 KDA 的输入投影。完成团队题 C1/C2 的同学可以使用这一行的实验结果作为后续分析的参考。

Editor's Note

4.1–4.3 从同一个 GEMM 出发，依次加入 tiling、TMA 和多级 pipeline。回顾梯子表时，重点不是最终的 cuBLAS 达成率，而是能够说明每一步优化减少了什么开销，以及新的瓶颈出现在哪里。

assignment01 Bonus 中 naive matmul 与 cuBLAS 之间的部分差距，现在已经可以从 Tensor Core、异步数据搬运和软件流水三个方面解释。进一步的 warp specialization、更大的 tile、epilogue 融合和 persistent kernel 等优化不在本作业要求范围内。

4.5 则说明 Tensor Core 的收益同样依赖矩阵形状。当 M 很小时，即使 kernel 本身使用了前面的优化方法，也不一定能够有效利用 Tensor Core。

5 低精度与 block scaling

降低数值精度可以换取更高的 Tensor Core 吞吐，但代价是可表示的动态范围变小。本模块先从 per-tensor scale 遇到 outlier 时的问题入手，再分析 block scaling 的代数约束，最后完成一条 NVFP4 量化通路，并通过融合实验观察低精度 kernel 的实际性能。

参考资料

课件 S094–S111 (fp8 格式: S096; per-tensor 与 outlier: S097–S099; block scaling 约束: S100–S101; fp4 与 NVFP4: S106–S108; SF 布局: S108); `cuda_fp4.h` 中的 `__nv_fp4x2_e2m1`; cuBLASLt 的 block-scaled FP4 matmul。格式约定见题面材料 `cuda/m5_lowprec/nvfp4_common.h`。

prob 5.1 (EXPERIMENT)

`kernels/quant_outlier.py`

观察 outlier 对 per-tensor scale 的影响。输入包含一万个 $[-1, 1]$ 均匀分布的元素，并额外加入一个值为 3000 的 outlier。补全脚本中的两个 TODO，完成 per-tensor E4M3 量化与反量化：

```
cd assignment02
uv run python kernels/quant_outlier.py
```

采样点 $x \approx$	0.5	0.1	0.01	0.005	3000
相对误差					

根据实验结果回答：

- (a) 去掉 outlier 后重新量化， $x \approx 0.5$ 处的误差变化多少倍？

- (b) 找出输入被量化为 0 的阈值，并写出该阈值与 scale 的关系式。
- (c) 改用 1×128 的 per-block scale 后，包含 outlier 的 block 与不包含 outlier 的 block 分别有什么变化？

prob 5.2 (DERIVE)

kernels/block_scale_sim.py

block scaling 的代数关键是沿着哪个方向分段 scale 乘积在 K 归约中才能保持常数。补全两个 fp64 模拟函数：

- `gemm_scale_per_row_col`: A 每行一个 scale, B 每个输出列一个 scale。scale 乘积在整个点积中不变，完整归约后只用乘回一次。
- `gemm_scale_along_k`: A 和 B 的 scale 每 128 个 K 元素改变一次。每个 K block 的 partial sum 要分别乘回该段的 scale 乘积，然后进行累加。

文件还提供了错误范例 `gemm_scale_along_k_one_restore`：它先把所有归一化 partial sum 相加，最后只乘回第一段的 scale。

判测：

```
cd assignment02
uv run pytest tests/test_block_scale.py
```

两个正确函数只要在 fp64 容差内与直接 GEMM 等价，不要要求 bit-exact（因为分段会改变浮点加法的分组）。然后回答：

- (a) 用两行代数式分别写出 row/column scale 为什么可以在整个点积外乘回，而 K-block scale 为什么必须逐段乘回。
- (b) NVIDIA CUTLASS 的 Blackwell SM100 GEMM 说明 把 A 的 scale 布局写成 $M \times [K/SV]$, B 写成 $N \times [K/SV]$, 每个 scale 负责连续的 16 或 32 个 K 元素。结合 GEMM 内循环、连续供数和 Tensor Core 指令语义，说明硬件为什么把 block scale 与 K 归约对齐。
- (c) DeepSeek-V3 的 weight 128×128 表示 scale 还在输出通道方向上共享，但每个组仍覆盖一段 K；NVFP4 则每 16 个 K 元素一组。结合 5.1(c) 的误差，说明粒度 16 相对粒度 128 有什么优势，又增加了多少 scale metadata 与供数复杂度。

prob 5.3 (FROM-SCRATCH)

cuda/m5_lowprec/

完成一条 NVFP4 量化通路。开始前先阅读 `nvfp4_common.h` 中给出的格式约定：

- 每个量化组包含 K 方向连续的 16 个元素；
- SF 使用 `amax / 6`，并以 e4m3 保存；
- SF 按 Tensor Core 消费要求的 swizzled 布局存放，字节偏移公式已经给出。

(a) **E2M1 编码** 在 `e2m1_encode.h` 中实现 host/device 通用的 E2M1 round-to-nearest-even 编码器。

判测会在 GPU 上使用 `cuda_fp4.h` 中的 `__nv_fp4x2_e2m1` 转换同一批候选值，包括所有中点、边界以及采样值，并与自己的编码结果逐位比较：

```
cd assignment02/cuda
make run/m5_lowprec/03a_encode_check
```

(b) **NVFP4 quant kernel** 在 `nvfp4_quant_kernel.h` 中实现 quant kernel, 完成

bf16 $\rightarrow$ e2m1 打包 + e4m3 SF + swizzled SF 布局

设备侧的 E2M1 转换直接使用 `__nv_fp4x2_e2m1`。

第一层判测要求输出逐 byte 与 host 参考结果相等; host 参考使用你在 5.3(a) 中实现的 E2M1 编码器生成。随后将量化结果原样交给 cuBLASLt 的 FP4 matmul, 检查生成的数据和 SF 布局能否被实际消费:

```
make run/m5_lowprec/03b_nvfp4_quant
make run/m5_lowprec/test_fp4_gemm
```

正确实现下, FP4 matmul 的 `maxrel` 约为 $4e-3$, 主要来自 bf16 输出的舍入误差。如果 SF 布局错位或 scale 对应到了错误的量化组, 通常会出现成块的明显数值错误, 而不是仅有小幅舍入误差。

(c) **Ceiling probe** 在 `03c_ceiling_probe.cu` 中实现一个 ceiling probe。它与 quant kernel 使用相同的访存模式, 但不执行量化计算, 只读取数据并通过简单 xor 后写回。

报告:

- ceiling probe 的 GB/s;
- quant kernel 的 GB/s;
- 两者的比值。

结合 Nsight Compute 判断 quant kernel 距离自己的访存上限还有多远, 以及剩余差距主要来自访存还是计算。

(d) **Optional** 使用 `tcgen05 kind::mxf4nvf4` 直接消费自己生成的 NVFP4 数据, SF 通过 TMEM 提供, 并与 cuBLASLt 的结果对拍。

Editor's Note

5.3(c) 用来区分量化 kernel 中的访存成本和数值转换成本。作为参考, 软件实现 E2M1 RN-even 时, 每个 kernel 约执行 18129 条 FSETP; 使用硬件转换后约为 1272 条 F2FP.E2M1, 同一 kernel 的时间从 $71.7\ \mu\text{s}$ 降到 $32.8\ \mu\text{s}$ 。Nsight Compute 中的瓶颈也从 SM 利用率 84.7% 的 compute-bound 转为 memory-bound。

这组结果说明, 当低精度转换由专用硬件完成后, 量化 kernel 可以更接近纯数据搬运的性能特征; 5.3(c) 的 ceiling probe 就用于衡量这一差距。

Hopper 上的 fp8 累加需要软件 promotion (S102-S105, DeepGEMM 使用双层累加循环), 而 Blackwell 进一步在硬件中支持 block scaling (S105)。本作业不单独设置对应题目, 但 5.2 中的 scale 分组约束是理解两者的共同基础。

prob 5.4(FROM-SCRATCH): 融合 rms_norm + NVFP4

实现融合的 rms_norm + NVFP4 kernel:

$$y = \text{rms_norm}(x) \cdot w$$

要求将结果直接量化为 NVFP4，不写回 bf16 中间结果。

本题以 vLLM fused kernel 清单中的 issue #25179，以及两个相关实现 PR #36413、#32957 为背景。相关实现的正确性没有问题，也确实将 kernel 数量从 2 个减少到了 1 个，但当时观察到的端到端收益仍处于 $\pm 1\%$ 左右的测量噪声内。本题要通过逐形状实验解释这部分收益去了哪里。

按理论字节量计算，两步实现约为 6.56 B/ele, 融合后约为 2.56 B/ele, 因此单纯从访存量估计可以得到约 $2.56\times$ 的理想加速。这个数字只是上限，实际结果需要通过实验验证。

程序已经提供两步基线、正确性检查和逐形状计时框架。测试覆盖：

- M 从 1 到 16384;
- $K \in \{4096, 7168, 8192\}$;
- 共十个测试形状。

需要完成三部分：

1. 实现融合 kernel，具体线程组织和 tile 结构不限；
2. 分别调优融合实现和两步基线，使两边都使用各自合理的配置后再比较；
3. 完成逐形状性能分析。

第二点必须认真处理。让两步基线处于明显不利的配置下得到的加速比没有比较意义，这也是相关上游 PR 的实验过程中暴露出的一个问题。

运行：

```
cd assignment02/cuda
make run/m5_lowprec/04_fused_rms_nvfp4
```

报告逐形状结果，包括：

- 实测加速比；
- 融合 kernel 相对 5.3(c) ceiling probe 的性能比例。

重点解释实测结果与 $2.56\times$ 理论上限之间的差距：不同 M 范围内主要受到什么因素限制，并给出 Nsight Compute 数据或计算结果作为依据。

本题不设置性能门槛。实现首先需要保证正确，评分重点放在实验设计和性能归因。

Optional：根据分析得到的主要瓶颈进行一次针对性优化，重新测试并更新表格。

prob 5.5 (CONCEPT)

比较 W4A16 + Marlin kernel (权重 int4、计算 fp16) 与 5.3 中的 NVFP4 GEMM。

根据课件 S109 的分类回答：

- (a) 两者分别属于存储量化还是计算量化？
- (b) 两种方法分别节省哪些资源：显存容量、显存带宽还是计算吞吐？
- (c) 在 4.5 中的小 batch decode 场景下，哪一类量化的收益更加直接？
每问用两到三句话回答。

6 TileLang 对照

前面的模块已经手动处理过 Tensor Core 指令、数据布局和数据搬运。本模块通过 TileLang 的 lowering 结果观察其中哪些工作可以交给 DSL 完成。

参考资料

课件 S112; assignment01 Module 7 (7.5 的表、7.6 的 `kernels/tilelang_matmul.py`); TileLang 文档中的 lowering/debug 部分。

prob 6.1 (EXPERIMENT)

使用 assignment01 的 `kernels/tilelang_matmul.py`, 或任意使用 `T.gemm` 的 kernel, 分别以 `sm_90a` 和 `sm_100a` 为 target 编译。本题只要求编译, 不需要实际使用 H 卡。

保存生成的 CUDA 源码与 lowering 输出, 并填写:

	sm_90a	sm_100a
选中的 Tensor Core 指令		
descriptor 在哪里、由谁生成		
smem swizzle 布局在哪一步确定		
数据由谁搬入 smem		

与 M2-M4 中的手写实现进行对照, 并回答:

- (a) 哪些硬件相关的决策已经由 DSL 自动完成?
- (b) 哪些参数仍然需要程序员决定, 例如 tile 尺寸和 stages?

最后在 assignment01 7.5 的“谁负责”表中补充一行:

Tensor Core 指令选择与供数布局

团队选做 (推荐)

2-4 人一组, 从 team/ 中的两个题目任选一个, 也可以全部完成:

- C1: FlashKDA 官方 kernel 当前使用 SM80 MMA, 分析迁移到 SM100 是否值得;
- C2: MiniMax M3 MSA 的小 batch decode 当前使用 Triton, 分析是否值得实现专用 kernel。

每道题分为三个阶段:

复现与测量→分析 (结论+ 证据) → 挑战

挑战阶段不要求一定得到正向加速。如果实验结果表明继续优化收益有限, 只要能够用数据说明为什么不值得继续做, 同样视为有效结论。

答辩时间为 10 分钟, 另有 5 分钟提问; 提问优先由选择另一道团队题的小组提出。具体要求见 `team/README.md`。

团队题不限制 AI 工具的使用。

提交

- **代码**：提交所有动手题的实现与判测输出；FROM-SCRATCH 题同时保留判测脚本的 PASS 记录。
- **报告**：包含纸面题解答、实验表格与性能归因，以及 DEBUG 题的现象记录和修改说明。所有实验数据注明使用的 GPU。
- **团队题**：提交代码、报告并完成答辩，具体要求见 `team/README.md`。